

	LEMBAR KERJA PRAKTIKUM ALGORITMA DAN PEMROGRAMAN II T.A. Genap 2025/2026	Nilai	Paraf Dosen

A. Identitas

Topik : Scope dan Multi-parameter function in python
 Kelas : 2C
 Nama : Muhammad Abdurrahman
 Tanggal Pengerjaan : 22/5/2026

B. Tujuan

1. Mahasiswa Mampu memahami konsep variable local dan global pada python serta mengerti konsep rekursif

C. Instruksi Pengerjaan

1. Cobalah semua *script* python yang ada pada modul materi.
2. Copy-paste *script* python pada box **Code**.
3. SS Output dimulai dari path tempat file.py disimpan. Contoh:

```
PS C:\Users\herup\OneDrive\Dokumen Pekerjaan\Algo2> python .\pertemuan2.py
60
```

D. Hasil Pengerjaan

1. Variable local: variable yang berada di dalam fungsi

Code:

```
#soal 1
print("--soal 1--")
def penjumlahan(x):
    bilangan = 7
    return x + 7

print(penjumlahan(4))
print(bilangan)
```

Output:

```
PS C:\Users\USER\Documents\04. Informatika\Semester 2> & "c:\Users\USER\Documents\04. Informatika\Semester 2\.\Informatika\Semester 2\Materi Kuliah\Alpro 2\Tugas\pertemuan12.py"
--soal 1--
11
Traceback (most recent call last):
  File "c:\Users\USER\Documents\04. Informatika\Semester 2\Materi Kuliah\Alpro 2\Tugas\pertemuan12.py", line 8, in <module>
    print(bilangan)
          ^^^^^^^^^
NameError: name 'bilangan' is not defined
PS C:\Users\USER\Documents\04. Informatika\Semester 2> |
```

Jelaskan:

Di program ini, variabel `bilangan` dibuat di dalam fungsi, jadi cuma bisa dipakai di dalam fungsi itu aja. Pas fungsi dipanggil dengan nilai 4, hasilnya 11. Tapi waktu `bilangan`

dipanggil di luar fungsi, muncul error karena variabel itu nggak dikenal di luar. Ini nunjukin kalau variabel lokal punya batas akses.

2. Variable di luar fungsi -1

Code:

```
#soal 2
print("--soal 2--")
# Contoh 1
bilangan = 2
def perkalian_bilangan(x):
    return x * bilangan

print(perkalian_bilangan(7))
```

Output:

```
--soal 2--
14
```

Jelaskan:

`bilangan` di sini dibuat di luar fungsi, jadi termasuk variabel global. Fungsi tetap bisa pakai variabel ini selama nggak mengubah nilainya. Makanya pas 7 dikali 2, hasilnya 14. Ini contoh kalau variabel global bisa dipakai di dalam fungsi tanpa masalah.

3. Variable di luar fungsi -2

Code:

```
#soal 3
print("--soal 3--")
# Contoh 2
def perkalian_bilangan2(x):
    bilangan = 5          # variabel lokal, menutupi yang global
    return x * bilangan

print(perkalian_bilangan2(7))
```

Output:

```
--soal 3--
35
```

Jelaskan:

Walaupun ada variabel global `bilangan`, di dalam fungsi dibuat lagi variabel dengan nama yang sama. Akibatnya, yang dipakai justru nilai yang di dalam fungsi, yaitu 5. Jadi hasilnya $7 \times 5 = 35$. Ini nunjukin kalau variabel lokal bisa “nutupin” variabel global.

4. Variable global dengan keyword 'global'

Code:

```
#soal 4
print("--soal 4--")
bilangan = 2
print(bilangan)

def return_bilangan():
    global bilangan
    bilangan = 5
    return bilangan

print(return_bilangan())
print(bilangan)
```

Output:

```
--soal 4--
2
5
5
```

Jelaskan:

Di sini dipakai keyword `global` supaya variabel `bilangan` yang ada di luar bisa diubah dari dalam fungsi. Awalnya nilainya 2, lalu di dalam fungsi diubah jadi 5. Setelah fungsi dijalankan, nilai di luar juga ikut berubah. Jadi, `global` dipakai kalau mau benar-benar mengubah variabel global dari dalam fungsi.

5. Kuis IMT

Code:

```
#soal 5
print("--soal 5--")
def hitung_imt(berat, tinggi):
    imt = berat / (tinggi ** 2)
    return imt

def kategori_imt(imt):
    if imt < 18.5:
        return "Berat badan kurang"
    elif imt >= 18.5 and imt < 25:
        return "Berat badan normal"
    elif imt >= 25 and imt < 30:
        return "Berat badan berlebih"
    else:
```

```

        return "Obesitas"

# input dari user
berat = float(input("Masukkan berat badan (kg): "))
tinggi = float(input("Masukkan tinggi badan (meter): "))

# proses
imt = hitung_imt(berat, tinggi)
kategori = kategori_imt(imt)

# output
print("Nilai IMT kamu:", round(imt, 2))
print("Kategori:", kategori)

```

Output:

```

--soal 5--
Masukkan berat badan (kg): 12
Masukkan tinggi badan (meter): 145
Nilai IMT kamu: 0.0
Kategori: Berat badan kurang

```

Jelaskan:

Program mulai dari minta input berat dan tinggi dari user. Setelah itu, nilai tersebut masuk ke fungsi `hitung_imt` buat dihitung IMT-nya. Hasilnya disimpan, lalu dikirim ke fungsi `kategori_imt` buat dicek masuk kategori apa lewat percabangan. Setelah itu, hasil IMT dan kategorinya ditampilkan ke layar.

6. Fungsi segitiga -1

Code:

```

#soal 6
print("--soal 6--")
def cek_segitiga(a, b, c):
    if a + b <= c:
        return False
    if b + c <= a:
        return False
    if c + a <= b:
        return False
    return True

print(cek_segitiga(1,1,1))
print(cek_segitiga(1,1,3))

```

Output:

```
--soal 6--  
True  
False
```

Jelaskan:

Program memanggil fungsi `cek_segitiga` dengan tiga angka. Di dalam fungsi, dicek kondisi pertama, kalau gagal langsung berhenti dan return False. Kalau lolos, lanjut ke kondisi berikutnya sampai semua dicek. Kalau semuanya aman, baru return True, lalu hasilnya ditampilkan.

7. Fungsi segitiga -2

Code:

```
#soal 7  
print("--soal 7--")  
def cek_segitiga(a, b, c):  
    if a + b <= c or b + c <= a or c + a <= b:  
        return False  
    return True  
  
print(cek_segitiga(1,1,1))  
print(cek_segitiga(1,1,3))
```

Output:

```
--soal 7--  
True  
False
```

Jelaskan:

Program memanggil fungsi, lalu langsung mengecek semua kondisi sekaligus pakai `or`. Kalau ada satu aja yang tidak memenuhi, fungsi langsung return False. Kalau tidak ada yang gagal, berarti semua aman dan return True.

8. Fungsi segitiga -3

Code:

```
#soal 8  
print("--soal 8--")  
def cek_segitiga(a, b, c):  
    return a + b > c and b + c > a and c + a > b  
  
print(cek_segitiga(1,1,1))  
print(cek_segitiga(1,1,3))
```

Output:

```
--soal 8--  
True  
False
```

Jelaskan:

Saat fungsi dipanggil, program langsung mengecek semua syarat pakai `and`. Kalau semuanya benar, hasilnya `True`, kalau ada yang salah langsung `False`. Hasilnya langsung dikembalikan tanpa langkah tambahan.

9. Kuis faktorial

Code:

```
#soal 9  
print("--soal 9--")  
def faktorial(n):  
    # Bilangan yang akan difaktorial harus lebih besar dari 0  
    if n < 0:  
        return None  
    # 0! dan 1! nilainya sama (1)  
    if n < 2:  
        return 1  
  
    hasil = 1  
    for i in range(2, n + 1): # dari 2 sampai n (inklusif)  
        hasil = hasil * i    # kalikan hasil dengan i  
    return hasil  
  
n = int(input("Masukkan nilai yang ingin di faktorial: "))  
print(n, "! =", faktorial(n))
```

Output:

```
--soal 9--  
Masukkan nilai yang ingin di faktorial: 34  
34 ! = 295232799039604140847618609643520000000
```

Jelaskan:

Program minta input angka dari user, lalu masuk ke fungsi `faktorial`. Di dalam fungsi dicek dulu kondisi awal, kalau aman lanjut ke perulangan. Angka dikali satu per satu mulai dari 2 sampai `n`, hasilnya disimpan terus sampai selesai. Setelah itu hasil faktorial dikembalikan dan ditampilkan

10. Kuis Fibonacci

Code:

```
#soal 10
print("--soal 10--")
def fibonacci(n):
    if n < 1:
        return None
    if n < 3:
        return 1

    elem_1 = elem_2 = 1
    hasil_jumlah = 0          # untuk menampung hasil penjumlahan 2
    elemen

    for i in range(n - 2):  # sudah ada 2 elemen awal, loop sisanya
        hasil_jumlah = elem_1 + elem_2  # Proses jumlah
        elem_1 = elem_2                # Tukar elemen: geser ke
        kanan
        elem_2 = hasil_jumlah          # Tukar elemen: simpan hasil
        baru

    return hasil_jumlah

# test
for i in range(1, 10):
    print(i, "->", fibonacci(i))
```

Output:

```
--soal 10--
1 -> 1
2 -> 1
3 -> 2
4 -> 3
5 -> 5
6 -> 8
7 -> 13
8 -> 21
9 -> 34
```

Jelaskan:

Program memanggil fungsi `fibonacci` untuk tiap angka. Di dalam fungsi, dicek kondisi awal, kalau bukan kondisi dasar, program mulai dari dua angka awal. Lalu dilakukan perulangan, setiap langkah menjumlahkan dua angka sebelumnya dan menggeser nilainya sampai dapat hasil. Setelah selesai, hasil dikembalikan dan ditampilkan.

11. Rekursif faktorial

Code:

```
#soal 11
print("--soal 11--")
def faktorial_rekursif(n):
    if n < 0:
        return None
    if n < 2:
        return 1
    return n * faktorial_rekursif(n - 1) # fungsi memanggil dirinya
    sendiri

n = int(input("Masukkan nilai faktorial (rekursif): "))
print(n, "! =", faktorial_rekursif(n))
```

Output:

```
--soal 11--
Masukkan nilai faktorial (rekursif): 32
32 ! = 263130836933693530167218012160000000
```

Jelaskan:

Program minta input, lalu masuk ke fungsi `faktorial_rekursif`. Kalau belum kondisi berhenti, fungsi akan panggil dirinya sendiri dengan nilai lebih kecil. Proses ini terus berulang sampai ketemu kondisi dasar, lalu hasilnya dikembalikan satu per satu ke atas sampai dapat hasil akhir

12. Rekursif fibonacci

Code:

```
#soal 12
print("--soal 12--")
def fibonacci_rekursif(n):
    if n < 1:
        return None
    if n < 3:
        return 1
    return fibonacci_rekursif(n - 1) + fibonacci_rekursif(n - 2) #
    memanggil diri sendiri 2x
```

```
# test
for i in range(1, 10):
    print(i, "->", fibonacci_rekursif(i))
```

Output:

```
--soal 12--
```

```
1 -> 1
```

```
2 -> 1
```

```
3 -> 2
```

```
4 -> 3
```

```
5 -> 5
```

```
6 -> 8
```

```
7 -> 13
```

```
8 -> 21
```

```
9 -> 34
```

Jelaskan:

Program memanggil fungsi `fibonacci_rekursif`. Kalau belum kondisi dasar, fungsi akan panggil dirinya dua kali, yaitu untuk $n-1$ dan $n-2$. Proses ini terus berulang sampai mencapai kondisi dasar, lalu hasilnya dijumlahkan dan dikembalikan sampai ke pemanggilan awal.